

CSCI-4208: Developing Advanced Web Applications

Week 5: Lecture 8
JavaScript - OOJS

Overview: JavaScript - OOJS

1. Objects
2. Classes
3. Class-based Inheritance
4. Prototypical Inheritance
5. Spread operator & Destructuring
6. Object methods

1. Objects: Overview

- Objects: In JS
- Objects: Creating Objects
- Objects: Adding Properties - (Key,Value pair)
- Objects: Accessing Properties
- Objects: Removing Properties
- Objects: Reference type

1. Objects: In JS

- Object' is a collection of related properties and behaviors.
 - properties → data values
 - behaviors → methods
- Objects are also dictionaries.
 - Property == (Key, Value) pair
 - Since functions are object types, they may be defined as a (key,value) pair
- Data Modeling - Objects are useful for representing a "thing"
 - Objects are quicker & easier to define & use for a single instance than by a class
 - Objects are the most abstract container of properties. Once created it can model any 'thing'
- Mutability - An object's properties are mutable
 - Properties may be added or deleted from an object at any time.
 - Objects may start empty or with a set of key-value pairs

1. Objects: Creating Objects

Three ways to generate basic objects.

- A. object initializer (literal notation)
- B. object constructor
- C. factory method

1. Objects: Creating Objects

A.) Object_INITIALIZER (literal notation)

Empty object $\rightarrow \{ \}$

```
> obj = {};  
< ▶ {}
```

Object with properties $\rightarrow \{ \text{key: value} \}$

```
> obj = { key1: "val1", key2: "val2"};  
< ▶ {key1: "val1", key2: "val2"}
```

1. Objects: Creating Objects

B.) Object's Constructor

use Object's constructor method to create a new empty object.

Empty Object → `new Object();`

```
> obj = new Object();  
< ▶ {}
```

1. Objects: Creating Objects

C.) **Factory Function**

A function that returns a configured object

Use Case: Literals for One object, Factories for Many objects

Factory functions offer the ability to easily produce object instances without diving into the complexities of classes.

1. Objects: Creating Objects

C.) Factory Function - Examples

Function Declaration

```
> function foo(x,y){  
    return { x:x, y:y};  
}  
  
foo(5,10);  
↵ ▶ {x: 5, y: 10}
```

Function Expression

```
> const foo = function (x,y){  
    return { x:x, y:y };  
}  
  
foo(5,10);  
↵ ▶ {x: 5, y: 10}
```

1. Objects: Creating Objects

C.) Factory Function - Examples

*Fat Arrow
function*

```
> const foo = (x,y)=> ({x:x, y:y})  
  
foo(5,10);  
  
< ▶ {x: 5, y: 10}
```

Explanation:

Use parentheses to encapsulate an object literal into a fat arrow function.

Why? The interpreter will parse the curly braces as a code block and not as an object literal

```
> const foo = (x,y)=> {x:x, y:y}  
✖ Uncaught SyntaxError: Unexpected token ':'
```

1. Objects: Creating Objects

C.) Factory Function - Default Parameters

Set default values in the parameter list

```
> const foo = (x=0,y=0)=> ({x:x, y:y})  
    foo();  
< ▶ {x: 0, y: 0}
```

1. Objects: Adding Properties - (Key, Value pair)

- Bracket notation

- Dictionary-styled accessor
- Property name may be any data type

```
> obj = new Object();  
obj[1] = "one";  
obj['foo'] = "bar";  
obj;  
◀ ▶ {1: "one", foo: "bar"}
```

- Dot notation

- Object-styled accessor
- Property name don't work with numbers

```
> obj = new Object();  
obj.foo = 'bar';  
obj;  
◀ ▶ {foo: "bar"}  
  
> obj.1 = 'one';
```

```
✖ Uncaught SyntaxError: Unexpected number
```

1. Objects: Adding Properties - (Key, Value pair)

- Legal Values:
Data or Functions

```
> obj = {};  
  obj.num = 10;  
  obj.foo = function(){ console.log('hi') };  
  obj;  
  
◀ ▶ {num: 10, foo: f}
```

-
- Composition
or Nesting Objects

```
> obj = {  
  nested: { foo: 'bar' }  
};  
  
◀ ▼ {nested: {...}} i  
  ▶ nested: {foo: "bar"}  
  ▶ __proto__: Object
```

1. Objects: Accessing Properties

- Bracket notation

- Dictionary-styled accessor
- Dereference using strings or numbers

```
> obj = { x:5, 10:'ten' };  
obj['x'];  
< 5  
  
> obj[10];  
< "ten"
```

- Dot notation

- Object-styled accessor
- Dereference as a property name

```
> obj = { x:5, 10:'ten' };  
obj.x;  
< 5  
  
> obj.10;  
✖ Uncaught SyntaxError: Unexpected number
```

- in operator

- boolean result if key is in object

```
> obj = {x:5, y:'blue'}  
'x' in obj;  
< true
```

1. Objects: Accessing Properties - Beware!

- Function invocations
vs Function definitions

```
> obj = { foo: x => x**2 };  
obj.foo           //returns the function object  
↵ x => x**2  
  
> obj.foo(5)      //invokes the function object  
↵ 25
```

-
- Exceptional Behaviors:
 - Accessing non-existing properties
does not cause an error.

```
> obj = {};  
obj.x;  
↵ undefined
```

1. Objects: Removing Properties

delete operator

```
> obj = {x:5, y:10};  
< ▶ {x: 5, y: 10}  
  
> delete obj.x;  
< true  
  
> obj  
< ▶ {y: 10}
```

Spread operator & destructuring

```
> obj = {x:5, y:10};  
  const {x, ...rest} = obj;  
  rest;  
  
< ▶ {y: 10}
```


1. Objects: Reference type

Objects are a reference data type An object passed into function could be mutated for all other references within the app.

```
> const bar = { x:"val" };  
   console.log("bar:", bar);
```

```
bar: ▶ {x: "val"}
```

```
> const foo = obj => obj.x = 0;  
  
foo(bar);  
console.log("bar:", bar);
```

```
bar: ▶ {x: 0}
```

1. Objects: Reference type

Deep Clone vs Shallow Copy

- Shallow Copy: (Note: doesn't work for nested ref types)

assign

```
> obj = { arr:[100], number:100 };  
copy = Object.assign({}, obj);  
◀ ▶{arr: Array(1), number: 100}  
  
> copy.arr[0] = 0  
console.log(obj.arr, copy.arr);  
▶ [0] ▶ [0] VM7  
  
◀ undefined  
  
> copy.number = 0;  
console.log(obj.number, copy.number);  
100 0 VM7
```

spread operator

```
> obj = {arr:[100], number:100 };  
copy = {...obj};  
◀ ▶{arr: Array(1), number: 100}  
  
> copy.arr[0] = 0;  
console.log(obj.arr, copy.arr);  
▶ [0] ▶ [0]  
  
◀ undefined  
  
> copy.number = 0;  
console.log(obj.number, copy.number);  
100 0
```

2. Classes: Overview

- **Classes: In JS**
 - mutable props
 - class vs instance
 - classes vs objects
- **Classes: class keyword**
 - declarations vs expressions
 - nesting classes
- **Classes: instance variables**
 - encapsulation - (private vs public)
- **Classes: constructor method**
- **Classes: instance methods**
- **Classes: this keyword**
- **Classes: static keyword**

2. Classes: In JS

- Classes serve as a blueprint to construct many different instances;
- instantiate a class into an instance with its constructor
- classes can have properties or instance variables
- classes can have behaviors or instance methods
- instances are mutable & properties may be add/removed at any time.

```
> class Foo{ }  
  const foo = new Foo();  
  foo.bar = "instances are mutable, like objects";  
  foo;  
  
◀ ▶ Foo {bar: "instances are mutable, like objects"}
```

2. Classes: class keyword

- `class`: The keyword for defining a class function.
-

- class declarations

- Hoisted

```
> class Foo { }  
  console.log(Foo);  
  
class Foo { }
```

-
- class expressions

- Not Hoisted

```
> const foo = class { };  
  console.log(foo);  
  
class { }
```

Anonymous

```
> const foo = class Foo { };  
  console.log(foo);  
  
class Foo { }
```

Named

2. Classes: class keyword

- **Inner class:** a classes defined within the scope of another class.

```
> class A {  
    B = class B{ };  
}  
◀ undefined
```

Inner classes must be defined as class expressions.

```
> class A {  
    class B{ }  
}  
✖ Uncaught SyntaxError: Unexpected identifier
```

2. Classes: instance variables

Definition: instance variables are the properties for an instance of a class

Declaring instance variable

- Put instance variables at top of class body, Declare the variables, do not initialize!

```
> class Foo{  
    x;  
}  
  
console.log(new Foo);  
► Foo {x: undefined}
```

Public: By default, data in classes & objects is not encapsulated (hidden).

Mutable: Technically you may declare instance variable anywhere in the class using the `this` keyword

2. Classes: instance variables

Encapsulation - Declare private instance fields with

1. *Public*
vs.
Private

```
> class Foo{  
  x;  
}  
  
console.log(Foo.x);  
undefined
```

public field:
external code may access

```
> class Foo{  
  #x;  
}  
  
console.log(Foo.#x);  
  
✖ Uncaught SyntaxError: Private field '#x'  
must be declared in an enclosing class
```

private field:
prevents external code from access

-
2. *Destructuring*: Private variables cannot be destructured to guarantee its encapsulation

```
> class Foo{  
  x;  
}  
  
const {x} = new Foo();  
← undefined
```

```
> class Foo{  
  #x;  
}  
  
const {#x} = new Foo();  
  
✖ Uncaught SyntaxError: Unexpected identifier
```


2. Classes: constructor method

- The constructor method initializes a class' instances properties.
 - The constructor method is invoked using the new keyword & the classname.
 - Use the this keyword to access the instance's properties.
-

constructor (no params)

```
> class Foo{
  #x;

  constructor(){
    this.#x = 10;
  }
}

foo = new Foo();
< ▶ Foo {#x: 10}
```

constructor (params)

```
> class Foo{
  #x;

  constructor(name){
    this.#x = name;
  }
}

foo = new Foo("bar");
< ▶ Foo {#x: "bar"}
```

constructor (default params)

```
> class Foo{
  #x;

  constructor(name="bar"){
    this.#x = name;
  }
}

foo = new Foo();
< ▶ Foo {#x: "bar"}
```

2. Classes: instance method

- instance method are declared in class body with a method name
- instance methods may be invoked from an instance using its name & params
- instance method objects may be accessed using just its name (no params)
- instance method may be invoked within the class using the `this` keyword

print invoked outside class

```
> class Foo{
  constructor(){
    this.x = 10;
  }
  print(){
    console.log(this.x);
  }
}
< undefined
> const foo = new Foo();
foo.print();
10
```

bar() invoked inside class

```
> class Foo{
  bar(){
    return "bar";
  }
  print(){
    const text = this.bar();
    console.log(text);
  }
}
< undefined
> const foo = new Foo();
foo.print();
bar
```

bar method object outside class

```
> class Foo{
  bar(){
    return "bar";
  }
  print(){
    const text = this.bar();
    console.log(text);
  }
}
< undefined
> const foo = new Foo();
const x = foo.bar;
x();
< "bar"
```

2. Classes: this keyword

1. A pointer that dereferences the object its in context of
2. this is relative based on scope
3. **Special Note:** When passing a reference of an instance method to another object or scope, you must bind its this pointer to the function object so you don't lose its context

```
> class Foo{
  constructor(){
    this.x = 10;
  }
  print(){
    console.log(this.x);
  }
}
< undefined
> const foo = new Foo();
const print = foo.print
< undefined
> print()
✖ ▶ Uncaught TypeError: Cannot read property
  'x' of undefined
    at print (<anonymous>:6:26)
    at <anonymous>:1:1
```

Wrong: Not binding object to instance method

```
> class Foo{
  constructor(){
    this.x = 10;
  }
  print(){
    console.log(this.x);
  }
}
< undefined
> const foo = new Foo();
const print = foo.print.bind(foo);
< undefined
> print()
10
```

Correct: binding object to its instant method

2. Classes: `this` keyword

`this` - Scoping rules

- In a method, *this* refers to the owner object
- Alone *this*, refers to the global object
- In a function, *this* refers to the global object
- In an event, *this* refers to the element that received the event

2. Classes: static keyword

1. Used to model a property or method of the class itself, not just an instance
 2. Static values are the same for all instances of the class
 3. Dereference static properties from the class name (not from the instance)
-

Correct

```
> class Foo{
    static x = "bar";
}
< undefined
> Foo.x
< "bar"
```

Wrong

```
> class Foo{
    static x = "bar";
}
< undefined
> const foo = new Foo();
    foo.x;
< undefined
```

2. Class Inheritance: overview

1. Class Inheritance: in JS
2. Class Inheritance: extends keyword
3. Class Inheritance: super keyword
4. Class Inheritance: method overriding
5. Class Inheritance: Polymorphism
 - a. instanceof operator

3. Class Inheritance: in js

Inheritance allows for a class to become a member of some other class inheriting all of its properties, behaviors and its type, while also allowing it to define its own properties, behavior, and sub-type.

Note: subclasses do not have access to superclasses private fields. Therefore, getter methods should be made in superclasses if they use private fields.

3. Class Inheritance: extends keyword

The extends keyword is used in class declarations or class expressions to create a class that is a child of another class.

Superclass A

Subclass B

- instantiate B
- A shows as its parent

B inherits props/methods of A

```
> class A {  
    print(){  
        console.log("A");  
    }  
}
```

```
class B extends A {  
}
```

```
b = new B();
```

```
< ▼ B {} ⓘ  
  ► __proto__: A
```

```
> b.print()  
A
```


3. Class Inheritance: super keyword

The **super** keyword is used to access and call functions on an object's parent or to invoke the parent class' constructor if it requires parameters.

Superclass A

- A requires data in its constructor

Subclass B

- B passes data to A
- instantiate B
- results

```
> class A {  
    #foo;  
    constructor(foo) {  
        this.#foo = foo;  
    }  
}
```

```
class B extends A {  
    constructor(){  
        super("B");  
    }  
}
```

```
b = new B();
```

```
< ▶ B {#foo: "B"}
```

3. Class Inheritance: method overriding

Method overriding allows a subclass to redefine the behavior of a method in the parent class.

A class may access both the parent classes method using **super** and its overridden method with **this**

```
> class A {  
  print () { console.log("A"); }  
}  
  
class B extends A {  
  print() {  
    console.log("B");  
  }  
  printAll(){  
    super.print();  
    this.print();  
  }  
}  
  
< undefined  
  
> const b = new B();  
b.printAll();  
  
A  
B
```

3. Class Inheritance: polymorphism

Motivation: When using inheritance for polymorphic behavior. You may opt to check if an instance is a member of a subclass in order to invoke a specific behaviors.

instanceof operator - checks whether an instance is a member of a particular class

```
> class A {}  
  
    class B extends A {}  
  
    const a = new A();  
    const b = new B();  
< undefined  
  
> a instanceof A  
< true  
  
> b instanceof B  
< true  
  
> b instanceof A  
< true
```

4. Prototypical Inheritance

1. In JS: objects may inherit properties and methods from a prototype property.
2. prototype property: a hidden property in a object that may used to build new object
3. inheriting with object's prototype

```
> foo = {prototype: 'foo', name: 'hello'}  
bar = Object.create(foo)  
  
< ▶ {}  
  
> Object.getPrototypeOf(bar)  
< ▶ {prototype: "foo", name: "hello"}  
  
> bar.name  
< "hello"
```

5. Spread operator & Destructuring

- Destructure assignment: unpack properties from objects, into distinct variables.
- Spread operator: used when multiple elements from an object need to be dereferenced & included together.

5. Spread operator & Destructuring: Objects

Object Destructuring

```
> let {a,b} = {a:10, b:20};  
< undefined  
  
> a  
< 10  
  
> b  
< 20
```

Object Destructuring & spread operator (*left*)

```
> let {a,b, ...rest} = {a:10, b:20, c:30, d:40};  
< undefined  
  
> a  
< 10  
  
> b  
< 20  
  
> rest  
< ► {c: 30, d: 40}
```

5. Spread operator & Destructuring: Objects

Object Destructuring (*right*):

```
> let a = {a:10, b:20};  
< undefined  
  
> let b = {...a, c:30};  
< undefined  
  
> b  
< ► {a: 10, b: 20, c: 30}
```

merge/add

5. Spread operator & Destructuring: Objects

Object Destructuring (*left + right*) :

```
> x = {a:10, b:20}  
y = {c: 30, d:30}  
const { a,c, ...z } = {...x, ...y};  
  
z;  
◀ ▶ {b: 20, d: 30}
```

mix & match

6. Object's static methods: Overview

Four categories of static methods defined in Object:

1. Object generators
2. Object transforms (Objects $\leftrightarrow$ Arrays)
3. Object encapsulations
4. Object metadata

6. Object - Static methods (Generators)

Object.create()

Creates a new object with the specified prototype object and properties.

```
> const obj = {name:"x"};
  obj.prototype = {type:"foo"};
  const fooObj = Object.create(obj);
< ▶ {type: "foo"}

> console.log(fooObj);
  console.log(fooObj.name);
  console.log(fooObj.prototype.type);

  ▶ {}
  x
  foo
```

Object.assign()

Copies the values of all enumerable own properties from one or more source objects to a target object.

```
> const target = { a: 1, b: 2 };
  const source = { b: 4, c: 5 };
  const returnedTarget = Object.assign(target, source);

  console.log(target);
  console.log(returnedTarget);

  ▶ {a: 1, b: 4, c: 5}
  ▶ {a: 1, b: 4, c: 5}
```

6. Object - Static methods (Transforms: Object → Array)

`Object.keys(o)`

Returns an array containing the names of all of the given object's **own** enumerable string properties.

```
> const obj = {x:0, y:1};  
   Object.keys(obj);  
◀ ▶ (2) ["x", "y"]
```

6. Object - Static methods (Transforms: Object $\rightarrow$ Array)

`Object.values(o)`

Returns an array containing the values that correspond to all of a given object's **own** enumerable string properties.

```
> const obj = {x:0, y:1};  
   Object.values(obj);  
  
< ▶ (2) [0, 1]
```

6. Object - Static methods (Transforms: Object $\rightarrow$ Array)

Object.entries(o) Returns an array containing all of the [key, value] pairs of a given object's **own** enumerable string properties.

```
> const obj = {x:0, y:1};  
   Object.entries(obj);  
  
< ▼ (2) [Array(2), Array(2)] ⓘ  
  ▶ 0: (2) ["x", 0]  
  ▶ 1: (2) ["y", 1]  
    length: 2  
  ▶ __proto__: Array(0)
```

6. Object - Static methods (Transforms: Array $\rightarrow$ Object)

Object.fromEntries(a) Returns a new object from an iterable of [key, value] pairs. (This is the reverse of **Object.entries**).

```
> Object.fromEntries([ ['x',0], ['y',1] ]);  
< ▶ {x: 0, y: 1}
```

6. Object - Static methods (Encapsulations)

`Object.preventExtensions(o)`

Prevents new properties from being added to an object (i.e. prevents future extensions to the object).

```
> const obj = { x:0 };  
Object.preventExtensions(obj);  
obj.x = 1;  
obj.y = 10;  
console.log(obj);  
  
▶ {x: 1}
```

`Object.isExtensible(o)`

Determines if extending of an object is allowed.

```
> const obj = { x:0 };  
Object.preventExtensions(obj);  
delete obj.x;  
console.log(obj);  
  
▶ {}
```

6. Object - Static methods (Encapsulations)

`Object.seal(o)`

Prevents other code from deleting properties of an object.

`Object.isSealed(o)`

Determines if an object is sealed.

```
> Object.isSealed(obj);
```

```
< true
```

```
> Object.isSealed({});
```

```
< false
```

```
> const obj = { x:0 };  
Object.seal(obj);  
obj.x = 1;  
obj.y = 10;  
console.log(obj);
```

```
▶ {x: 1}
```

```
> const obj = { x:0 };  
Object.seal(obj);  
delete obj.x;  
console.log(obj);
```

```
▶ {x: 0}
```


6. Object - Static methods (Encapsulations)

`Object.freeze(o)`

Freezes an object. Other code cannot delete or change its properties

```
> const obj = { x:0 };  
Object.freeze(obj);  
obj.x = 1;  
obj.y = 10;  
console.log(obj);  
  
▶ {x: 0}
```

`Object.isFrozen(o)` Determines if an object was frozen.

```
> Object.isFrozen(obj)  
< true  
  
> Object.isFrozen({})  
< false
```

6. Object - Static methods (Object metadata)

`Object.defineProperty()`

Adds the named properties described by the given descriptors to an object.

```
> const obj = {};  
   const props = { 'foo':{value:'bar'}, 'spam':{value:'eggs'} }  
   const result = Object.defineProperty(obj, props);  
   console.log("obj: ",obj,"result:",result);  
  
obj:   ► {foo: "bar", spam: "eggs"} result: ► {foo: "bar", spam: "eggs"}
```

`Object.defineProperty()`

Adds the named property described by a given descriptor to an object.

```
> const obj = {};  
   const result = Object.defineProperty(obj, 'foo', {value:'bar'});  
   console.log("obj: ",obj,"result:",result);  
  
obj:   ► {foo: "bar"} result: ► {foo: "bar"}
```

6. Object - Static methods (Object metadata)

`getOwnPropertyDescriptor()`

Returns a property descriptor for a named property on an object.

```
> obj
< ▶ {foo: "bar", spam: "eggs"}
> Object.getOwnPropertyDescriptor(obj, 'foo');
< ▶ {value: "bar", writable: false, enumerable: false, configurable: false}
```

`getOwnPropertyDescriptors()`

Returns an object containing all own property descriptors for an object.

```
> obj
< ▶ {foo: "bar", spam: "eggs"}
> Object.getOwnPropertyDescriptors(obj)
< ▼ {foo: {...}, spam: {...}} ⓘ
  ▶ foo: {value: "bar", writable: false, enumerable: false, configurable: false}
  ▶ spam: {value: "eggs", writable: false, enumerable: false, configurable: false}
  ▶ __proto__: Object
```

6. Object - Static methods (Object metadata)

getOwnPropertyNames() Returns an array containing the names of all of the given object's **own** enumerable and non-enumerable properties.

getOwnPropertyNames

vs.

keys

```
> obj
< ▶ {foo: "bar", spam: "eggs"}
> Object.getPrototypeOf(obj);
< ▶ (2) ["foo", "spam"]
```

```
> Object.getPrototypeOf(['a', 'b'])
< ▶ (3) ["0", "1", "length"]
> Object.keys(['a', 'b'])
< ▶ (2) ["0", "1"]
```

6. Object - Static methods (Object metadata)

getPrototypeOf()

Returns the prototype (internal `[[Prototype]]` property) of the specified object.

```
> obj
< ▾ {prototype: {...}, foo: "bar", spam: "eggs"} ⓘ
  ► prototype: {type: "foobar"}
    foo: "bar"
    spam: "eggs"
  ► __proto__: Object

> const x = Object.create(obj);
  Object.getPrototypeOf(x);
< ► {prototype: {...}, foo: "bar", spam: "eggs"}

> x
< ► {}
```

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Lab 6

'JavaScript - OOJS'

Homework 6

Browser App - JS Paint